Admins

Those who have not collected HW1/2/3/Midterm scripts can collect them outside Steven's office (COM2-3-37)

HW3-2014-SS.pdf and T1-2014-SS.pdf will be uploaded tomorrow (Thu 17 Apr) after last tutorial

Last minute consultation window:
Thu 17 Apr, Thu 24 Apr, Fri 25 Apr

CS3230 Lecture 13

Summary Lecture

(stevenhalim@gmail.com)

Lecture Topics

- Revisit the second half of CS3230
- Various means to handle NPC problems including new stuffs: Better Complete Search, Local Search, Parallel/Distributed computing
- Advertisement of algorithm related stufs @ SoC

if all you have is a hammer,
everything looks like a nail

The First Half of CS3230

1. Algorithms are beautiful, HW1
2. Algorithms need to be analyzed
3. Divide and Conquer paradigm,
   analysis of recursive algorithm (Master's theorem)
4. Sorting, the $\Omega(n \log n)$ bound for comparison sort,
   and how to beat that
5. Order Statistics, Augmenting DS, PA1, HW2
6. Amortized Analysis, 3 methods:
   aggregate, accounting, potential

It is not all about P

Before 2nd Half of CS3230	Now
All CS problems that you know have time complexity O(f(n)) when f(n) is a ***low order poly-nomial***, i.e. n, n log n, n^2, etc Examples: Sorting, Binary Search, Order Statistics, Graph Traversal, MST, SSSP, etc	There are many CS problems that belongs to the class of **NP-Complete** Examples: CIRCUIT-SAT, SAT, 3-CNF-SAT, CLIQUE, VERTEX-COVER, HAMILTONIAN-CYCLE, SUBSET-SUM

NP-Completeness

NP-Complete theory deals with **decision problems**

The optimization version of an NP-Complete problem is *not easier* than the corresponding decision version

- **KNAPSACK inputs:** Consider a set of **n** items where the i-th item is of weight $\mathbf{w_i}$ and of value $\mathbf{v_i}$, also suppose the maximum load of your knapsack is **W** and $\mathbf{w_i} \leq \mathbf{W}$ for all i
- **KNAPSACK (optimization version)**: Find a subset of items $\mathbf{S} \subseteq \{1, 2, \ldots, n\}$ **<u>s.t. $\Sigma i \in S\ v_i$ is maximized</u>** while $\mathbf{\Sigma_{i \in S} w_i} \leq \mathbf{W}$
- **KNAPSACK (decision version)**: Find a subset of items $\mathbf{S} \subseteq \{1, 2, \ldots, n\}$ **<u>s.t. $\Sigma i \in S\ v_i \geq K$ for a value K</u>** while $\mathbf{\Sigma_{i \in S} w_i} \leq \mathbf{W}$

Class NP

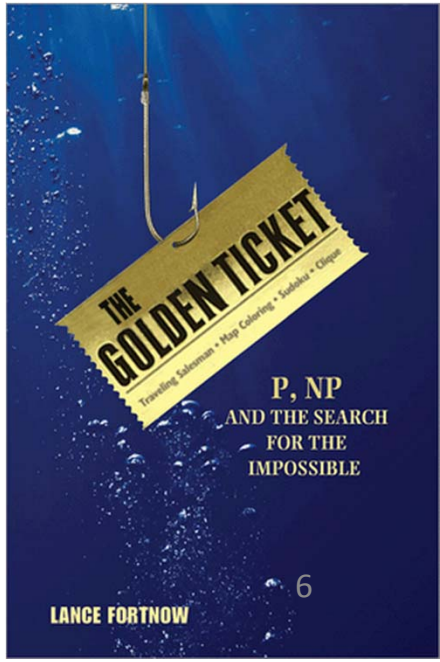
The problems in **NP (Non-deterministic Polynomial)** can be **verifiable in polynomial time**

- If the problem has a solution and we have a **certificate** of the solution, we can verify its correctness in polynomial time

NOTE: We can choose the format of the certificate

- The verifier algorithm is not responsible for PRODUCING the certificate, it just VERIFIES the certificate in polynomial time

Open problem: P = NP (or P ≠ NP)



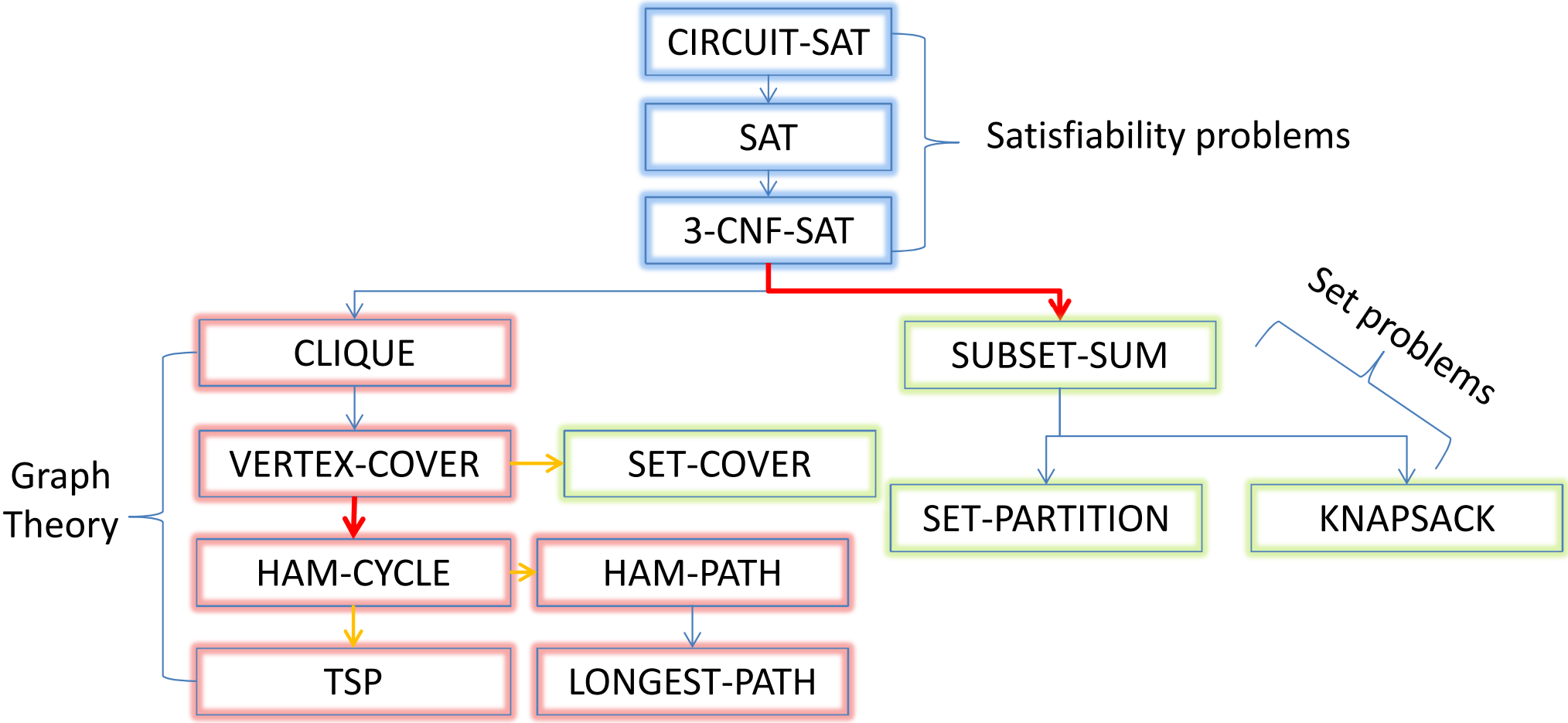
NP-Complete

To prove a problem **L** is NP-Complete,
we need to show that:

1. $L \in NP$ (the easy part, see the previous slide)
2. L is NP-hard, i.e. another NP-Complete problem L' can be reduced to L in polynomial time, i.e. $L' \leq_p L$
   a) Show that the reduction is correct, i.e. show ←→ (iff)
   b) Show that the reduction algorithm runs in polynomial time

Polynomial-time reduction/transformation/conversion ($A \leq_p B$) is a very important concept and applicable for P problems too (not just for proving NP-hardness)

NPC Roadmap of CS3230 (so far)



There are many other NPC problems, some in different fields

Dealing with an NPC problem (1.1)

Solve very small instances optimally with Naïve Search

Naïve Search:

Simply try all possibilities… *in super polynomial time*

- Complete Search (or sometimes known as brute force) is a "universal" tool to solve NPC problems as computer is much faster than human, but it may not be the best tool for all cases

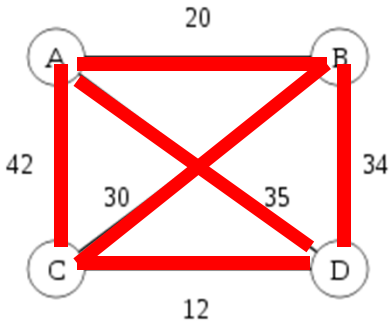
Dealing with an NPC problem (1.2)

Solve small instances optimally with better Complete Search

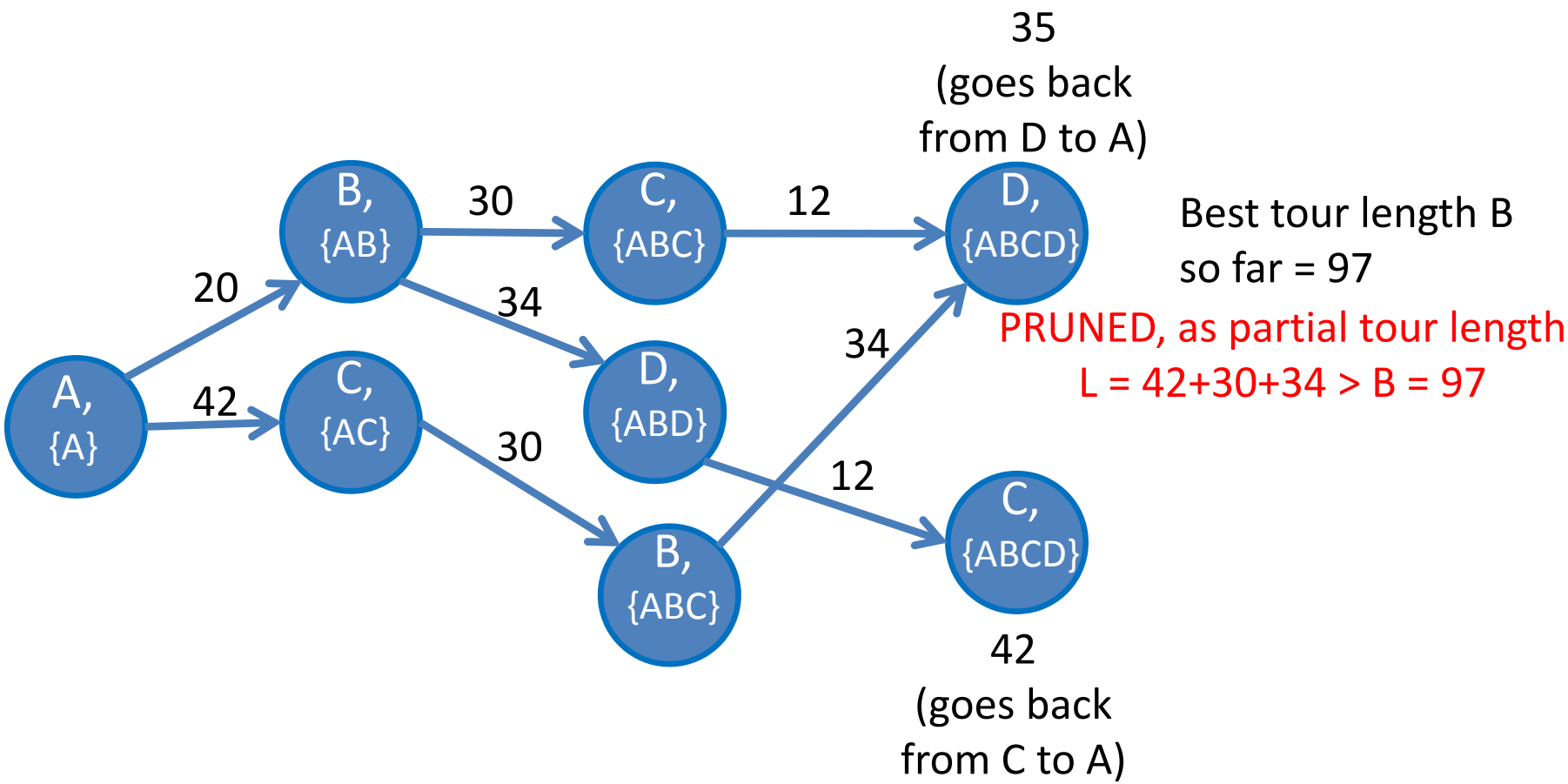
We can design **better Complete Search** algorithm

One example: Branch and Bound for TSP

- In backtracking routine (that search for all O(N!) tours), we compare the current partial tour length L with the best tour length B found so far, if L > B, we can prune the partial tour!
- Note: This is just a simple bounding function and there are more sophisticated bounding functions for TSP



A Small Preview



Dealing with an NPC problem (2.1)

Identify special case(s)/variant(s) that is/are still in P

Some special case(s)/variant(s) of NPC problems may still have polynomial solution(s)

NPC	P
3-CNF-SAT	2-CNF-SAT, DNF-SAT
TSP	BITONIC-TSP
HAM-CYCLE/PATH	EULER-CYCLE/PATH
LONGEST-PATH	LONGEST-PATH in DAG
VERTEX-COVER	VERTEX-COVER on TREE
This list is not exhaustive	

Dealing with an NPC problem (2.2)

Identify special case(s)/variant(s) that is/are still in P

2-CNF-SAT is in **P** as we can change each 2-CNF clause ($a \vee b$) into two implications $\neg a \rightarrow b$ and $\neg b \rightarrow a$

We can then build the implication graph from such implication edges and if there is no variable and its negation in the same Strongly Connected Component (SCC), the 2-CNF formula is satisfiable

Make sure you understand the details DFS tree and of Kosaraju's SCC finding algorithm (based on DFS)

Dealing with an NPC problem (3.1)

Solve small instances optimally with DP

Dynamic Programming (DP) is a powerful problem solving technique and applicable if the problem has:

1. **Optimal Substructure** (the hard part)
   - In short, you can describe the problem as a recursive function with some parameters that relates the problem to its subproblems, i.e. you can define the recursive cases and the base cases
2. **Overlapping Subproblems** (usually not too hard to notice)
   - In short, you notice that some (mostly many) subproblems are recomputed more than once
   - If this property is not found, DP is “applicable” but useless

Dealing with an NPC problem (3.2)

Solve small instances optimally with DP

There are two ways to implement DP:

1. **Top-Down** (a.k.a. memoization technique)
   - Use memo table initially filled with "dummy values"
   - Implemented using recursion that checks if the subproblem
     a) Has been computed before → just O(1) lookup in the memo table
     b) Has not been computed before → compute it ONCE and store the result in the correct position in the memo table, overriding the dummy value
2. **Bottom-Up**
   - Use proper DP table initially filled with base case values
   - Usually implemented using proper (nested) loop(s) to process the subproblems in the topological order of the subproblems DAG
     - This is to ensure that whenever we need to compute the value of a problem, all the relevant subproblems have been optimally computed

Dealing with an NPC problem (3.3)

Solve small instances optimally with DP

To analyze the space complexity and time complexity of a basic (unoptimized) DP algorithm is simple:

- **Space complexity** (i.e. DP table size) is simply the maximum number of different subproblems that we can possibly have (i.e. the number of vertices in the subproblems DAG)
- **Time complexity** is simply the time needed to compute one subproblem * the number of different subproblems (i.e. the number of edges in the subproblems DAG) because we ensure that we only compute each subproblem **ONCE**

Of course, we can "optimize" our DP algorithms…

Dealing with an NPC problem (3.4)

Solve small instances optimally with DP

DP is usually applied to **P** problems (to turn a seemingly super-polynomial algorithm into a polynomial one) or **<u>sometimes</u>** to **NPC** problems (to enlarge the reach of **small** instances that are still solvable optimally in reasonable amount of time)

Problem	Naïve Brute Force	Dynamic Programming
CUT-ROD, P	$O(n * 2^n)$, exponential	$O(n^2)$, polynomial
KNAPSACK, NPC	$O(n * 2^n)$, exponential	$O(n\mathbf{W})$, pseudo-polynomial
TSP, NPC	$O(n!)$, factorial	$O(n^2 * \mathbf{2^n})$, still exponential
A few other examples in tutorials and in HW3		

Dealing with an NPC problem (4.1)

Get good enough results with Greedy (heuristic)

Greedy Algorithm is another powerful problem solving technique that is applicable if the problem has:

1. **Optimal Substructure**
   - This is the same as with DP
2. **Greedy Choice**
   - **Locally optimal** choice lead to **global optimal** in some **P** problems
     - However, we need to prove such greedy choice (usually via **'exchange argument'**)
   - Greedy choice makes the algorithm **fast** (usually iterative)

Dealing with an NPC problem (4.2)

Get good enough results with Greedy (heuristic)

Greedy algorithm is usually applied to **P** problems (which may already have *another* polynomial algorithm into a faster polynomial algorithm)

Note: A problem that accepts Greedy algorithm can also be solved with Dynamic Programming or even Complete Search, although overkill

Dealing with an NPC problem (4.3)

Get good enough results with Greedy (heuristic)

When applied to **NPC** problems, we will **NOT** get correct/optimal result (otherwise P = NP) but we can sometimes get **good enough** solutions in reasonable amount of time; we call this Greedy **heuristic** (or Greedy **approximation algorithm**)

Problem	DP	Greedy
ACTIVITY-SELECTION, P	$O(n^3)$, polynomial	O(n), faster polynomial
HUFFMAN-CODE, P	-	O(n log n)
TSP, NPC	$O(n^2 * 2^n)$, exponential	$O(n^2)$, polynomial, **suboptimal**
SET-COVER, NPC	-	fast (log |X|+1)-**approximation**
A few other examples in tutorials and in HW3		

Dealing with an NPC problem (5.1)

Get good enough results with Approximation Algorithm

An algorithm is an approximation algorithm if it is applied to (the optimization version of) an NPC problem and has a proven approximation ratio ρ**(n)**

- **max(C/C*, C*/C)** $\leq \rho(n)$, we aim to minimize ρ**(n)**
- Sometimes, the approximation ratio is independent of **n**

Not all NPC problems have approximation algorithm, some can be proven to be difficult to approximate :O

Note: Polynomial Time Approximation Scheme (PTAS) is an advanced technique not yet covered this semester

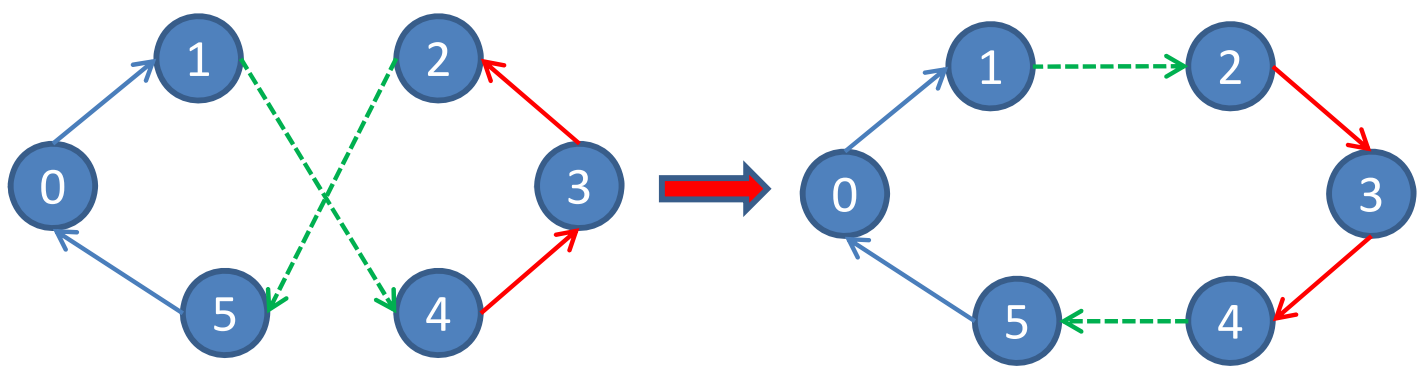
Dealing with an NPC problem (6.1)

Get good enough results with Local Search

Introducing new search paradigm: **Local Search**, again applied to a well known NPC problem: TSP

Definition:

- 2-Opt/2-Exchange/Edge swap **local** move/operation
  - Take two edges (a, b) and (c, d) currently in the tour s-…-a-b-X-c-d-…-s
  - Swap them to edges (a, c) and (b, d) to get tour s-…-a-c-**rev(X)**-b-d-…-s



Dealing with an NPC problem (6.2)

Get good enough results with Local Search

Definition (continued):

- 2-Opt **Neighborhood**
  - Given a current tour of N cities, we can choose up to ${}_NC_2 = O(N^2)$ edges
    - We pick two different vertices a and c in the tour and also neighbor(a) $\neq$ c
    - The corresponding edges are (a, neighbor(a)) and (c, neighbor(c))
  - If the current tour T has cost C, swapping two edges of T using 2-Opt local move will create another tour T' with cost C'
    - If C' > C, we call this an "uphill move"
    - If C' == C, we call this a "sidestep or plateau move"
    - If C' < C, we call this a "downhill move"

Dealing with an NPC problem (6.3)

Get good enough results with Local Search

Local Search for TSP starts from any starting tour **T** (remember Greedy Nearest Neighbor Heuristic?) and keep picking the best downhill move repeatedly to produce **T→T'→T''→**until it runs out of option…

At this point, we say that the Local Search hits a **Local Optima**, which may or may not be the **Global Optima**

- Most of the time, it is not the Global Optima…
- What should we do?

Dealing with an NPC problem (6.4)

Get good enough results with Local Search

Solution: We simply re-do the local search again ☺, but **using a different "starting tour"**

The Iterated Local Search (ILS) algorithm (pseudocode):

```
BT = T = GreedyNNH(); // start with any valid tour
while (total run time is still < max allowed time)
  N(T) = 2OptNeighborhood(T); // O(N^2) possible 2-Opt moves
  NextT = Best(N(T)); // complexity depends on implementation
  if cost(NextT) < cost(T) // have at least 1 downhill move
    if cost(NextT) < cost(BT)
      BT = NextT; // keep the best tour found so far
  else if cost(NexT) >= cost(T) // all plateau/uphill moves
    T = Perturb(T); // restart with new T, details next
```

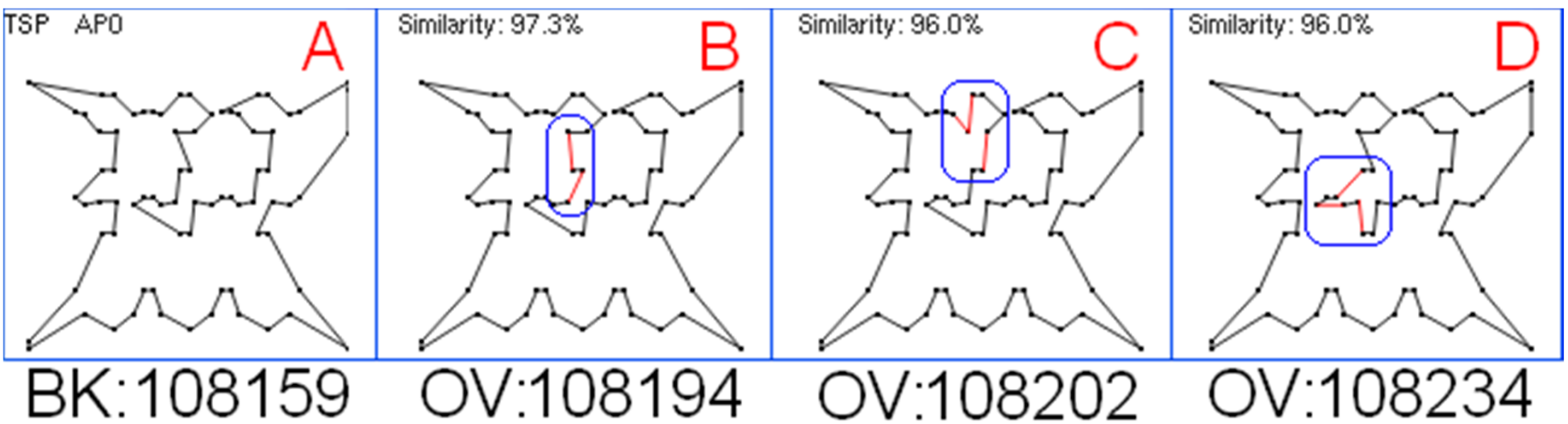
Dealing with an NPC problem (6.5)

Get good enough results with Local Search

Question: What is the best (a good) way for Perturb(T)?

The **"Big Valley"** property: Good solutions are close (in structure) with (one of) the optimal solution

- Example on pr76 instance (76 cities), $2^{76}*76^2 = 4*10^{26}$ with DP

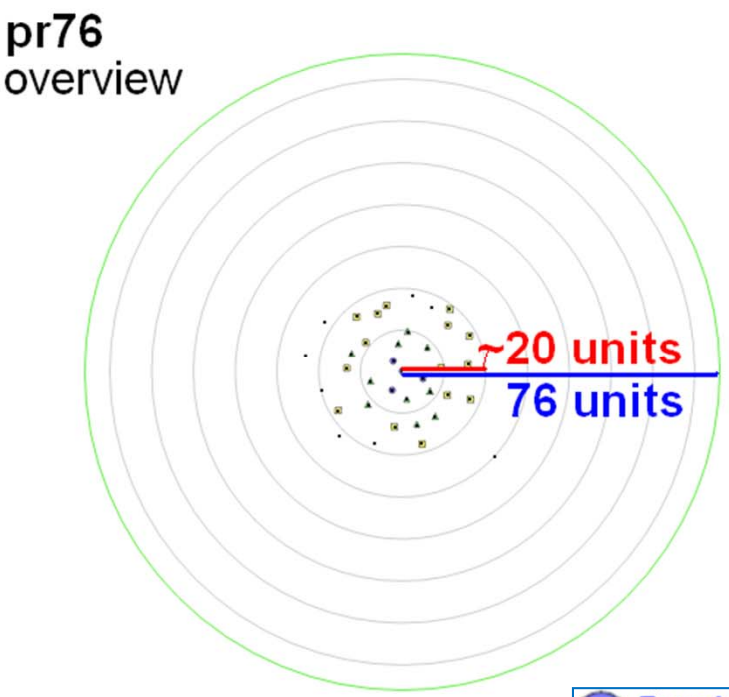


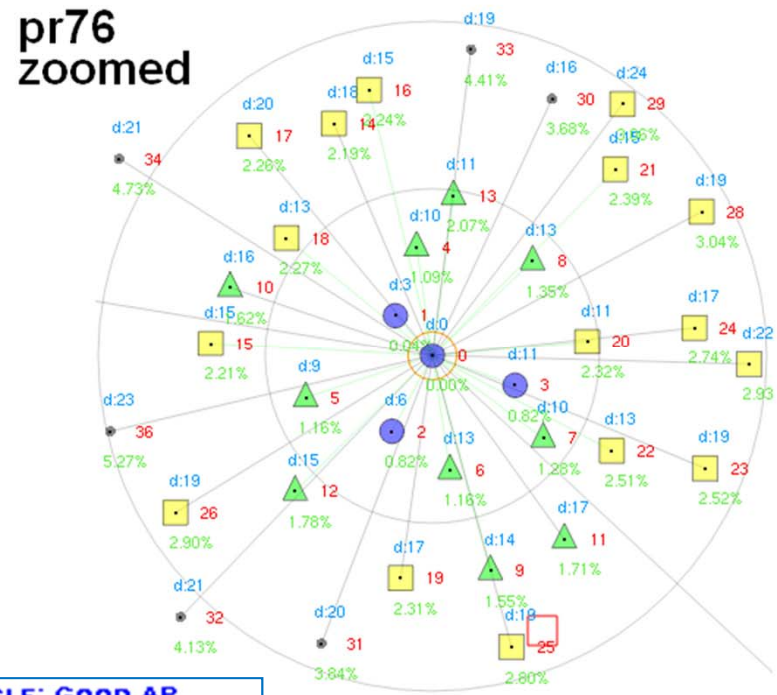
Dealing with an NPC problem (6.6)

Get good enough results with Local Search

Looking at this phenomenon from a different angle

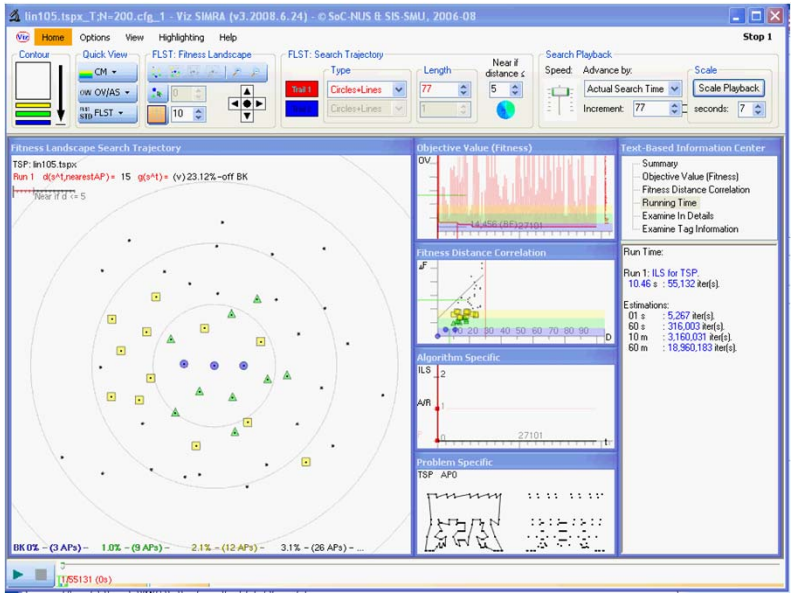
Intuition for good (local) search: **Focus** the search there





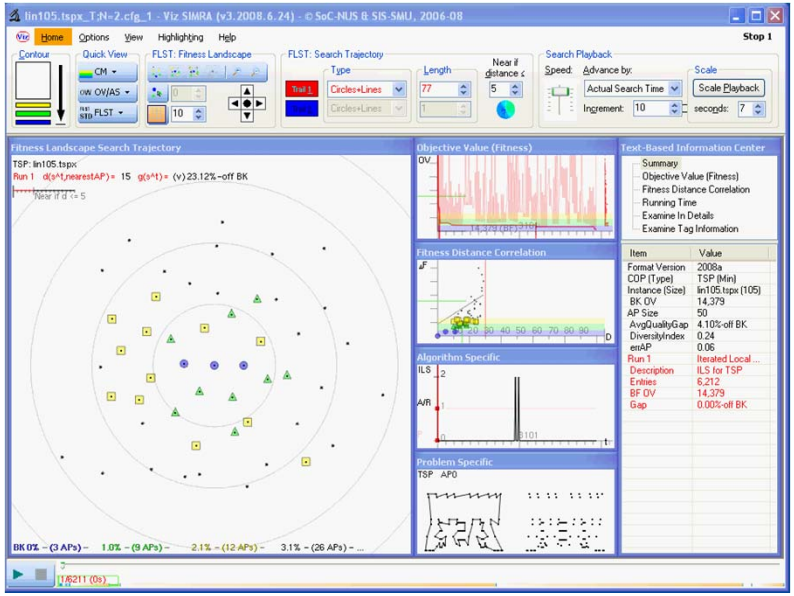
Dealing with an NPC problem (6.7)

Get good enough results with Local Search



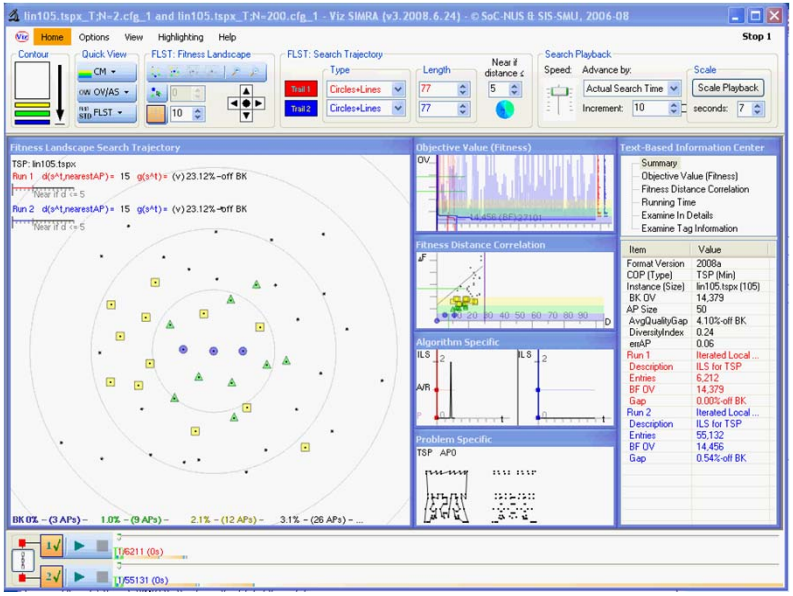
ILS_{BTR}
(more or less what we discussed just now)

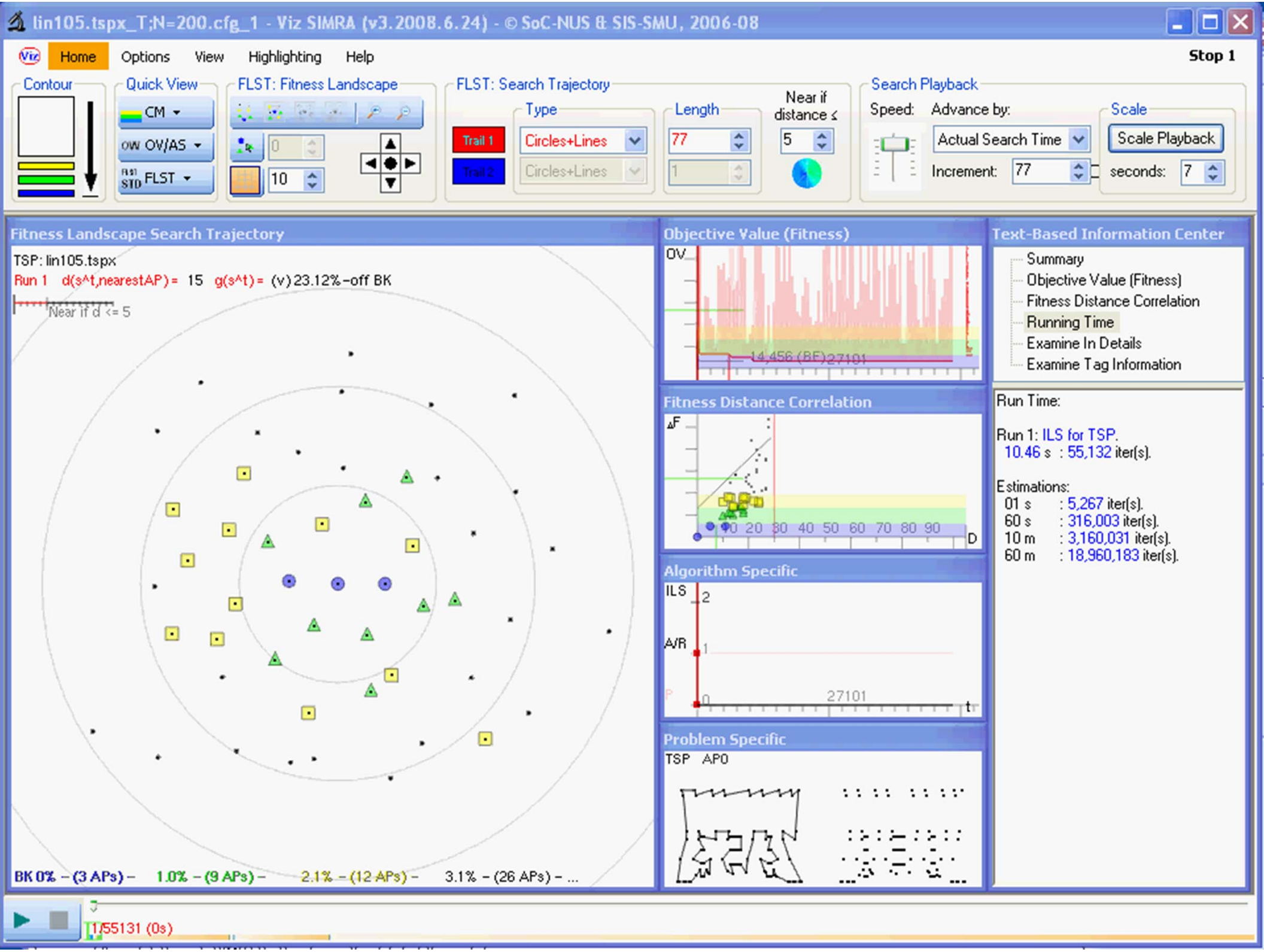
Steven's Local Search **visualization tool** (Phd Thesis, 2009)



ILS_T
(details omitted)

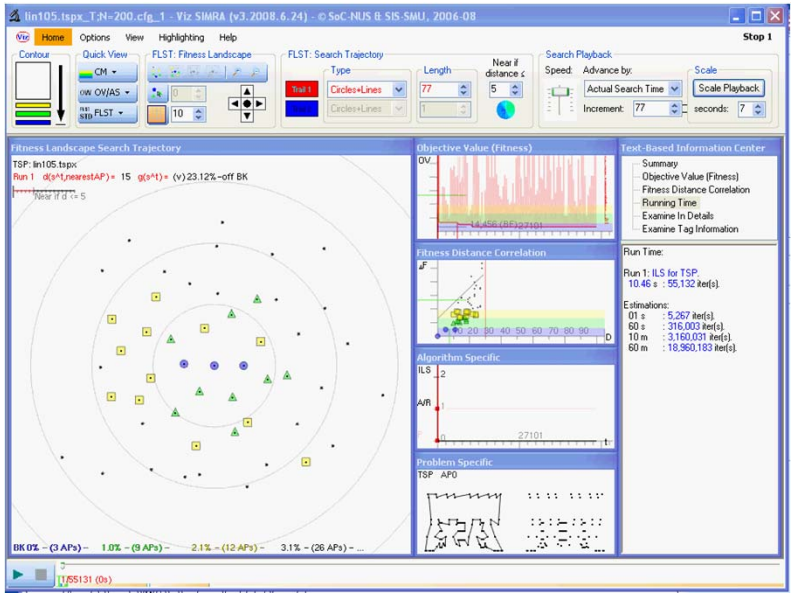
Better view:
Visual Comparison of ILS_{BTR} versus ILS_T




lin105.tspx_T;N=200.cfg_1 - Viz SIMRA (v3.2008.6.24) - © SoC-NUS & SIS-SMU, 2006-08
Home
Options
View
Highlighting
Help
Stop 1
Contour
Quick View
CM
OW OV/AS
FLST
FLST: Fitness Landscape
10
FLST: Search Trajectory
Trail 1
Trail 2
Type
Circles+Lines
Circles+Lines
Length
77
1
Near if distance ≤
5
Search Playback
Speed:
Advance by:
Actual Search Time
Increment:
77
Scale
Scale Playback
seconds:
7
Fitness Landscape Search Trajectory
TSP: lin105.tspx
Run 1 d(s^t,nearestAP) = 15 g(s^t) = (v) 23.12%-off BK
Near if d <= 5
BK 0% - (3 APs) - 1.0% - (9 APs) - 2.1% - (12 APs) - 3.1% - (26 APs) - ...
Objective Value (Fitness)
OV
14,456 (BF) 27101
Fitness Distance Correlation
ΔF
0 20 30 40 50 60 70 80 90
D
Algorithm Specific
ILS
2
A/R
1
P
0
27101
t
Problem Specific
TSP AP0
Text-Based Information Center
Summary
Objective Value (Fitness)
Fitness Distance Correlation
Running Time
Examine In Details
Examine Tag Information
Run Time:
Run 1: ILS for TSP.
10.46 s : 55,132 iter(s).
Estimations:
01 s : 5,267 iter(s).
60 s : 316,003 iter(s).
10 m : 3,160,031 iter(s).
60 m : 18,960,183 iter(s).
1/55131 (0s)

Dealing with an NPC problem (6.7)

Get good enough results with Local Search

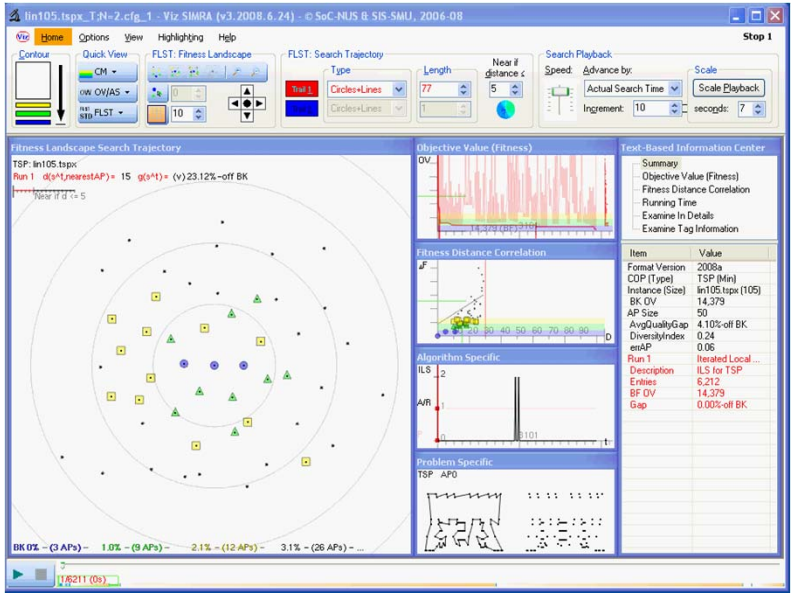


ILS_{BTR}

(more or less what we discussed just now)

Steven's Local Search **visualization tool**

(Phd Thesis, 2009)

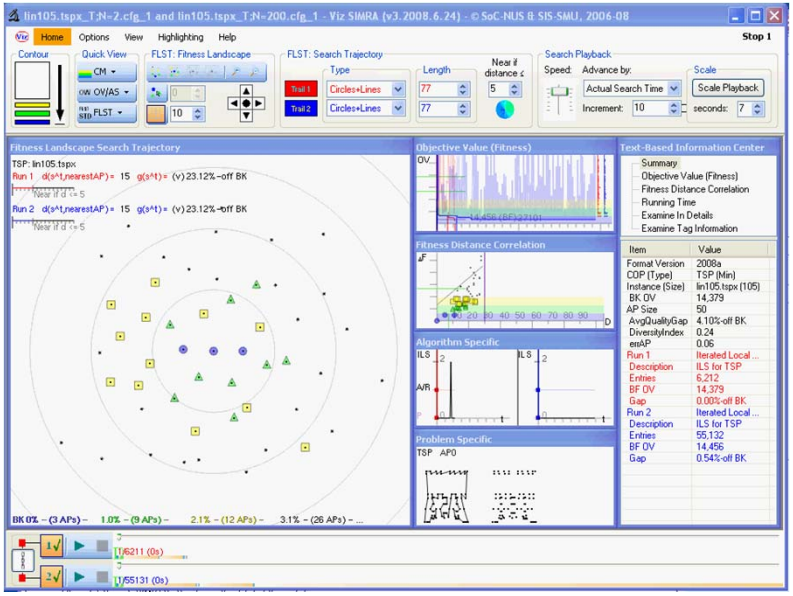


ILS_T

(details omitted)

Better view:

Visual Comparison of ILS_{BTR} versus ILS_T



lin105.tspx_T;N=2.cfg_1 - Viz SIMRA (v3.2008.6.24) - © SoC-NUS & SIS-SMU, 2006-08

Home | Options | View | Highlighting | Help

Stop 1

Contour

Quick View: CM, OW OV/AS, FLST

FLST: Fitness Landscape: 0, 10

FLST: Search Trajectory: Trail 1, Trail 2; Type: Circles+Lines, Circles+Lines; Length: 77, 1; Near if distance ≤ 5

Search Playback: Speed; Advance by: Actual Search Time; Increment: 10; Scale: Scale Playback; seconds: 7

Fitness Landscape Search Trajectory

TSP: lin105.tspx

Run 1 d(s^t,nearestAP) = 15 g(s^t) = (v) 23.12%-off BK

Near if d <= 5

BK 0% - (3 APs) - 1.0% - (9 APs) - 2.1% - (12 APs) - 3.1% - (26 APs) - ...

Objective Value (Fitness)

OV

14,379 (BF) 3101

Fitness Distance Correlation

ΔF

20 30 40 50 60 70 80 90

D

Algorithm Specific

ILS 2

A/R 1

P 0

3101

t

Problem Specific

TSP AP0

Text-Based Information Center

- Summary
- Objective Value (Fitness)
- Fitness Distance Correlation
- Running Time
- Examine In Details
- Examine Tag Information

Item	Value
Format Version	2008a
COP (Type)	TSP (Min)
Instance (Size)	lin105.tspx (105)
BK OV	14,379
AP Size	50
AvgQualityGap	4.10%-off BK
DiversityIndex	0.24
errAP	0.06
Run 1	Iterated Local ...
Description	ILS for TSP
Entries	6,212
BF OV	14,379
Gap	0.00%-off BK

1/6211 (0s)

lin105.tspx_T;N=2.cfg_1 and lin105.tspx_T;N=200.cfg_1 - Viz SIMRA (v3.2008.6.24) - © SoC-NUS & SIS-SMU, 2006-08

Home | Options | View | Highlighting | Help

Stop 1

Contour

Quick View: CM, OW OV/AS, FLST

FLST: Fitness Landscape: 0, 10

FLST: Search Trajectory

	Type	Length	Near if distance ≤
Trail 1	Circles+Lines	77	5
Trail 2	Circles+Lines	77	

Search Playback

Speed: Advance by: Actual Search Time

Increment: 10

Scale: Scale Playback

seconds: 7

Fitness Landscape Search Trajectory

TSP: lin105.tspx

Run 1 d(s^t,nearestAP) = 15 g(s^t) = (v) 23.12%-off BK

Near if d <= 5

Run 2 d(s^t,nearestAP) = 15 g(s^t) = (v) 23.12%-off BK

Near if d <= 5

BK 0% - (3 APs) - 1.0% - (9 APs) - 2.1% - (12 APs) - 3.1% - (26 APs) - ...

Objective Value (Fitness)

OV

14,456 (BF) 27101

Fitness Distance Correlation

ΔF

0 20 30 40 50 60 70 80 90 D

Algorithm Specific

ILS 2 A/R P 0 t

ILS 2 0 t

Problem Specific

TSP AP0

Text-Based Information Center

- Summary
- Objective Value (Fitness)
- Fitness Distance Correlation
- Running Time
- Examine In Details
- Examine Tag Information

Item	Value
Format Version	2008a
COP (Type)	TSP (Min)
Instance (Size)	lin105.tspx (105)
BK OV	14,379
AP Size	50
AvgQualityGap	4.10%-off BK
DiversityIndex	0.24
errAP	0.06
Run 1	Iterated Local ...
Description	ILS for TSP
Entries	6,212
BF OV	14,379
Gap	0.00%-off BK
Run 2	Iterated Local ...
Description	ILS for TSP
Entries	55,132
BF OV	14,456
Gap	0.54%-off BK

1 √ 1/6211 (0s)

2 √ 1/55131 (0s)

Dealing with an NPC problem (6.8)

Get good enough results with Local Search

I had good **empirical** results ☺

That (and of course other results) gave me my PhD ☺

Wilcoxon signed-ranks test[8] detects that the results of the baseline algorithm ILS_{BTR} is *clearly* better than ILS_{RW} (14 pairs, $p < 0.05, T = 0.0, V = 25$) and the results of the tweaked algorithm ILS_{T} is *significantly* better than the baseline algorithm ILS_{BTR} (14 pairs, $p < 0.05, T = 19.0, V = 25$).

Test Instances

Instance	Iters	*BK* OV	ILS_{RW}		ILS_{BTR}		ILS_{T}	
			$\bar{x}$	σ	$\bar{x}$	σ	$\bar{x}$	σ
berlin52	2704	7542	2.44	0.95	0.49	0.98	**0.00**	0.00
st70	4900	675	2.37	0.56	0.91	0.71	**0.22**	0.30
rat99	9801	1211	3.69	0.82	0.66	0.83	**0.21**	0.23
kroB100	10000	22141	2.01	0.59	0.42	0.48	**0.11**	0.16
rd100	10000	7910	3.32	0.56	0.96	0.80	**0.15**	0.21
eil101	10201	629	5.48	0.60	1.39	0.74	**1.21**	0.74
pr107	11449	44303	8.74	4.62	**3.56**	2.43	4.45	2.08
bier127	16129	118282	2.46	0.40	0.29	0.19	**0.24**	0.12
ch130	16900	6528	3.02	0.59	0.87	0.41	**0.41**	0.36
u159	25281	42080	3.92	0.85	0.84	0.68	**0.22**	0.27
d198	39204	15780	11.49	2.90	**1.36**	0.68	1.37	0.52
tsp225	50625	3916	5.55	0.66	0.70	0.60	**0.66**	0.42
gil262	68644	2378	5.70	0.57	**0.87**	0.42	1.00	0.38
a280	78400	2579	8.87	1.07	1.39	0.85	**1.18**	0.44
		Average ⇒	4.93		1.05		**0.82**	

Table 7.2: ILS Variants Results

Dealing with an NPC problem (7.1)

Enlarge solvable instance sizes with **more workers**

Today, individual CPU is not that “expensive”

Some (parts of the) algorithms that we learn so far can be parallelized or run of distributed computers

Not all algorithms can be parallelized though

Final Exam Overview

- The weightage of the final exam is 40%
- There are 5 questions, 10+10+10+5+5 = 40 marks
- The weightage of questions involving first half and second half of CS3230 are 10% and 30%, respectively
- The average CA marks of CS3230 students as of Tuesday 15 April 2014 is around 42.3 / 55.0 (~77%)
  - Most of you will pass this module ☺
- There are some challenging questions in the paper
- Some subquestions have hints, a few are of 0/1 type

Final Exam Feedback Plan

- Modal answer and marking scheme **DRAFT** will be uploaded to IVLE on Saturday, 26 April 2014, ~3pm, after the exam ends
  - You can view it instantly after exam while your memory is still fresh
  - Estimate your score by yourself, **assume we are a strict graders**
- Give us (myself + Prof Leong) a few days of grading
- Friday, 2 May 2014, 10am, I will upload the final exam scores to IVLE gradebook and update the modal answer + marking scheme with annotations of common mistakes in final exam
- Friday, 2 May 2014, 11am-5pm will be the only time window for **viewing** your CS3230 final exam script, Steven's office (COM2-3-37), no other viewing time window is planned

Advertisement

If you like CS3230, you will (probably also like):

- CS3233 – Competitive Programming (Steven Halim)
- CS4231 – Parallel and Distributed Algorithms (Yu Haifeng)
- CS4232 – Theory of Computation (Sanjay Jain)
- CS5234 – Combinatorial and Graph Algorithms (Seth Gilbert)
- CS6234 – Advanced Algorithms (Rahul Jain)
- Do research in algorithms
  (UROP, FYP, MSc by research, or PhD)
  Current trend: Big Data, Online Algorithm, …

THE END